

CS599 期末大作业报告

课程名称	企业级应用软件设计与开发
项目名称	CodeSage — 基于 Agentic AI 的智能代码工程师 Agent
项目方向	方向一：Agentic AI 原生开发
学号	2025302936
姓名	龙志文
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 18 日

目录

CS599 期末大作业报告	1
目录	错误!未定义书签。
一、选题背景与设计思想	4
1.1 问题定义	4
1.3 项目价值	4
1.4 技术路线	4
二、Specs 规格文档	5
2.1 Product Spec (产品规格)	5
2.2 Architecture Spec (架构规格)	6
2.3 API Spec (API 规格)	7
三、系统架构与设计	8
3.1 核心架构图	8
3.2 Agent 交互流程	9
3.3 核心设计决策	10
四、关键实现与代码展示	11
4.1 Agent 核心循环	11
4.2 工具定义 — FileReader	11
4.3 配置文件 — 环境变量	12
4.4 双引擎审查器	13
4.5 对话记忆持久化	14
4.6 AI IDE 使用截图	15
五、测试与评估	15
5.1 功能测试	15

5.2 Agent 行为评估	16
5.3 Benchmark	16
5.4 Demo 截图	17
六、系统升级与扩展	18
6.1 可扩展架构	18
6.2 版本迭代计划	19
6.3 AI 能力演进路径	19
七、课程总结	19
7.1 个人收获	19
7.2 工程思维转变	19
7.3 课程建议	20

一、选题背景与设计思想

1.1 问题定义

现代软件开发中，代码审查与质量保障面临两个核心挑战：

- 1. **人工审查效率低**：开发者提交 Pull Request 后需逐行阅读代码，大型 PR 审查耗时长，且人类注意力有限，易遗漏隐蔽问题；
- 2. **审查标准不一致**：不同审查者经验差异大——初级工程师可能只关注格式，资深工程师覆盖安全/性能/架构，缺乏统一质量基线；

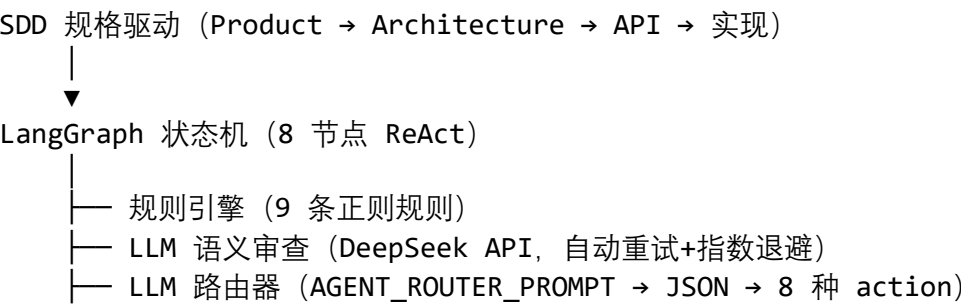
1.3 项目价值

CodeSage 构建具备完整工程闭环的 AI Agent 系统：

- **自然语言交互**：用日常语言描述需求（"审查这个项目""修复上次的严重问题""回退修改"），LLM 路由器自动识别 8 种操作意图并路由执行；
- **双引擎代码审查**：规则引擎（9 条正则，0.1 秒级）+ LLM 语义审查协同工作，互补盲区；
- **Bug 自动修复**：LLM 分析 → 提取修复代码 → 自动备份到 .codesage/backups/ → 写回源文件，全程可追踪可回退；
- **单元测试生成**：自动生成 pytest 测试写入 test_*.py，形成"审查→修复→测试"质量闭环；
- **跨会话记忆**：ConversationMemory 记录审查历史、文件操作追踪和对话上下文，持久化到 .codesage/memory.json，重启后自动恢复。

1.4 技术路线

采用 SDD 方法论（Specification-Driven Development），围绕 Agentic AI 原生开发六大核心要素：



- └─ FileReader 工具层 (读写+集中备份+模糊搜索)
- └─ ConversationMemory (对话+文件操作追踪+JSON 持久化)

技术选型: Python 3.11+ + LangGraph 0.2 + DeepSeek API + Rich 13 终端 UI。

二、Specs 规格文档

2.1 Product Spec (产品规格)

产品定义: CodeSage 是基于 AI Agent 的智能代码工程师助手, 面向代码审查、Bug 修复、测试生成三大环节。

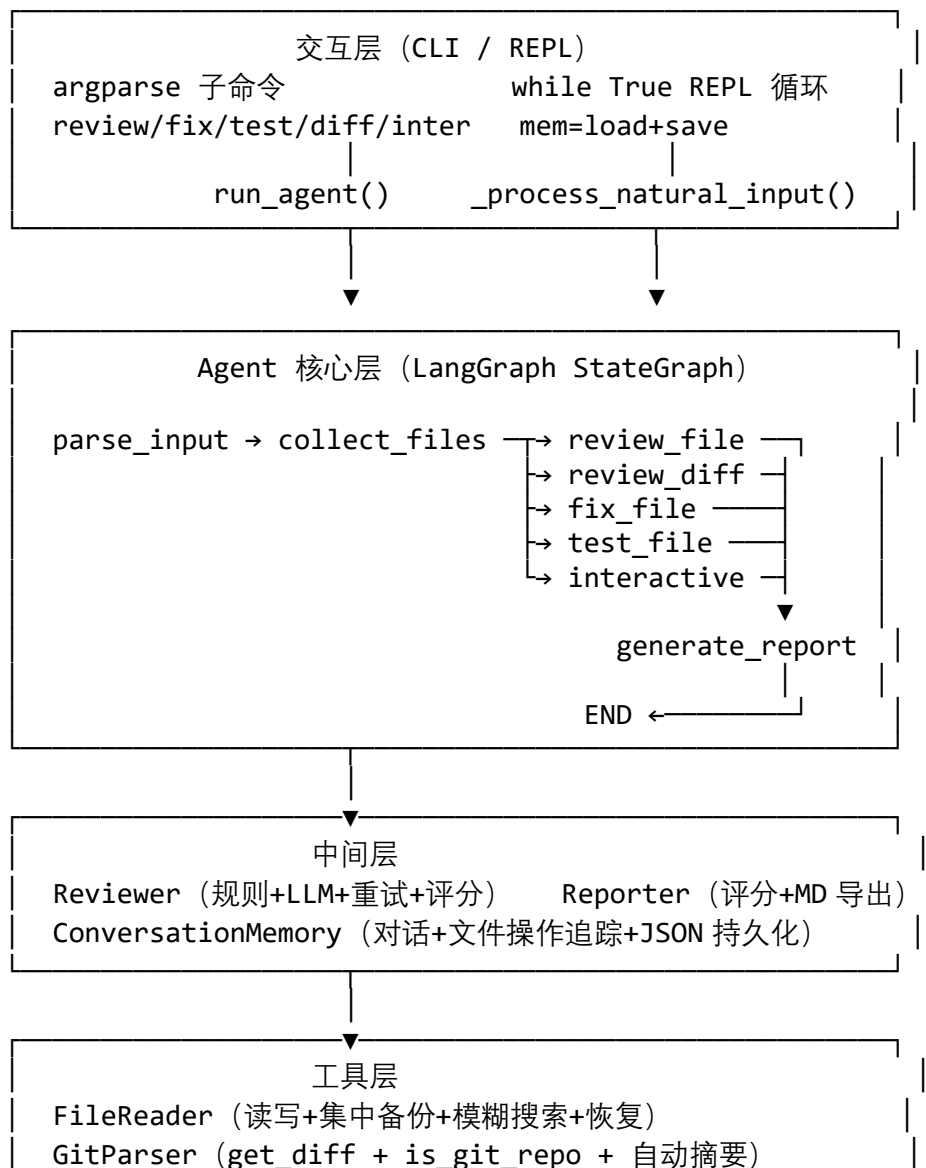
功能	描述	触发方式	优先级
自然语言交互	LLM 路由器识别 review/fix/test/diff/refactor/stats/ undo/chat 共 8 种意图	CLI REPL 自然语言输入	P0
单文件审查	规则引擎正则匹配 + LLM 语义分析, 输出结构化报告含评分	/review file 或自然语言	P0
目录递归审查	glob 递归扫描, 逐文件规则引擎 即时输出, 上限 MAXDIRFILES(=10)	/review dir/ 或自然语言	P0
Bug 修复	LLM 分析 → 提取修复代码 → 备份 原文件 → 写回	/fix file 或自然语言	P0
单元测试生成	LLM 生成 pytest 测试 → 写入 test_*.py	/test file 或自然语言	P0
Git diff 审查	解析 git diff --cached, LLM 深度 分析 + 变更文件规则审查	/diff 或自然语言	P1
代码重构建议	LLM 分析代码异味、SOLID 原则	自然语言触发	P1
版本回退	从 .codesage/backups/ 恢复历史 版本, 支持交互式选择	/undo <file> 或自然语言	P1
对话记忆	审查历史 + file_ops 追踪 + JSON 持久化, 重启恢复	自动	P1
模糊路径匹配	路径不存在时递归搜索候选项, 交互式选择	自动触发	P2
目录统计	递归扫描源文件, 统计行数和语言分布	自然语言触发	P2
报告导出	审查报告保存为 Markdown	-o report.md	P2

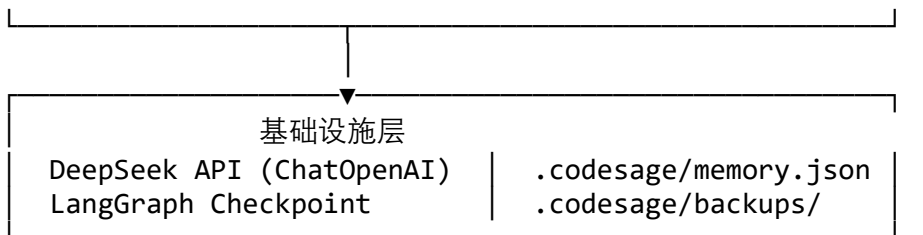
用户故事:

1. 开发者提交前输入"审查这个项目" → Agent 递归审查 src/ 下所有源文件，逐文件输出；
2. 审查发现硬编码 API Key → 输入"帮我修复" → Agent 备份原文件并写回修复代码；
3. 修复后发现逻辑有误 → 输入"回退 main.py" → Agent 列出所有备份版本供选择恢复；
4. 修改完成 → 输入"生成测试" → Agent 从记忆找到最后修改文件并写 test_*.py。

2.2 Architecture Spec (架构规格)

系统分层架构





AgentState 数据模型

```
class AgentState(TypedDict):
    messages: Annotated[list[Any], add_messages] # 消息流
    target: str # 操作目标
    target_type: str # review_file|review_dir|fix|test|diff|query
    issues: list[dict] # 规则引擎发现的问题
    llm_review_text: str # LLM 自然语言输出
    diff_content: str # Git diff 文本
    diff_summary: str # Git diff 摘要
    current_file: str # 当前处理文件
    files_processed: list[str] # 已处理文件列表
    fixed_code: str # 修复后代码
    original_code: str # 修复前代码
    backup_path: str # 备份路径
    fix_instruction: str # 用户修复指令
    score: float # 综合评分
    done: bool # 完成标记
    error: str # 错误信息
```

2.3 API Spec (API 规格)

LangGraph 节点接口

节点	输入 → 输出	后续路由
parse_input	messages → target_type	→ collect_files (固定)
collect_files	target, targettype → currentfile, files_processed	→ reviewfile / fixfile / testfile / reviewdiff / interactive / generate_report (条件路由)
review_file	currentfile → issues, llmreview_text, score	→ generate_report (固定)
review_diff	diffcontent, diffsummary → issues, llmreviewtext	→ generate_report (固定)
fix_file	currentfile, fixinstruction → issues, backuppath, _writeresult	→ generate_report (固定)

节点	输入 → 输出	后续路由
test_file	currentfile → llmreviewtext, _testwrite_result	→ generate_report (固定)
generate_report	全量 issues + llm_text → AIMessage	→ END
interactive	messages, issues → AIMessage	→ END

审查规则接口

```
@dataclass
class ReviewRule:
    rule_id: str # "SEC-001"
    category: Category # SECURITY/QUALITY/BEST_PRACTICE/PERFORMANCE
    severity: Severity # CRITICAL/WARNING/SUGGESTION
    title: str
    description: str
    suggestion: str
    patterns: dict[str, str] # {"python": r"...", "java": r"..."}

```

审查结果接口

```
@dataclass
class ReviewIssue:
    severity: Severity; category: Category
    rule_id: str; file_path: str; line: int | None
    title: str; description: str; suggestion: str
    source: str # "rule"/"llm"/"system"

```

LLM Router API

输入记忆上下文 + 用户输入 → LLM 返回 JSON:

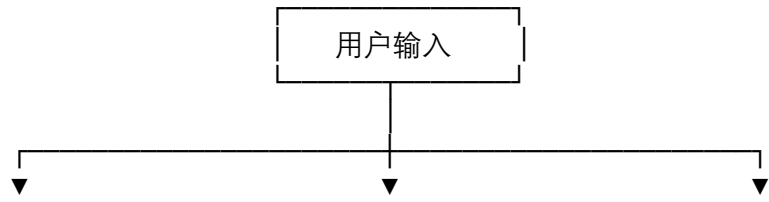
```
{"action": "review", "file": "src/main.py", "instruction": ""}
```

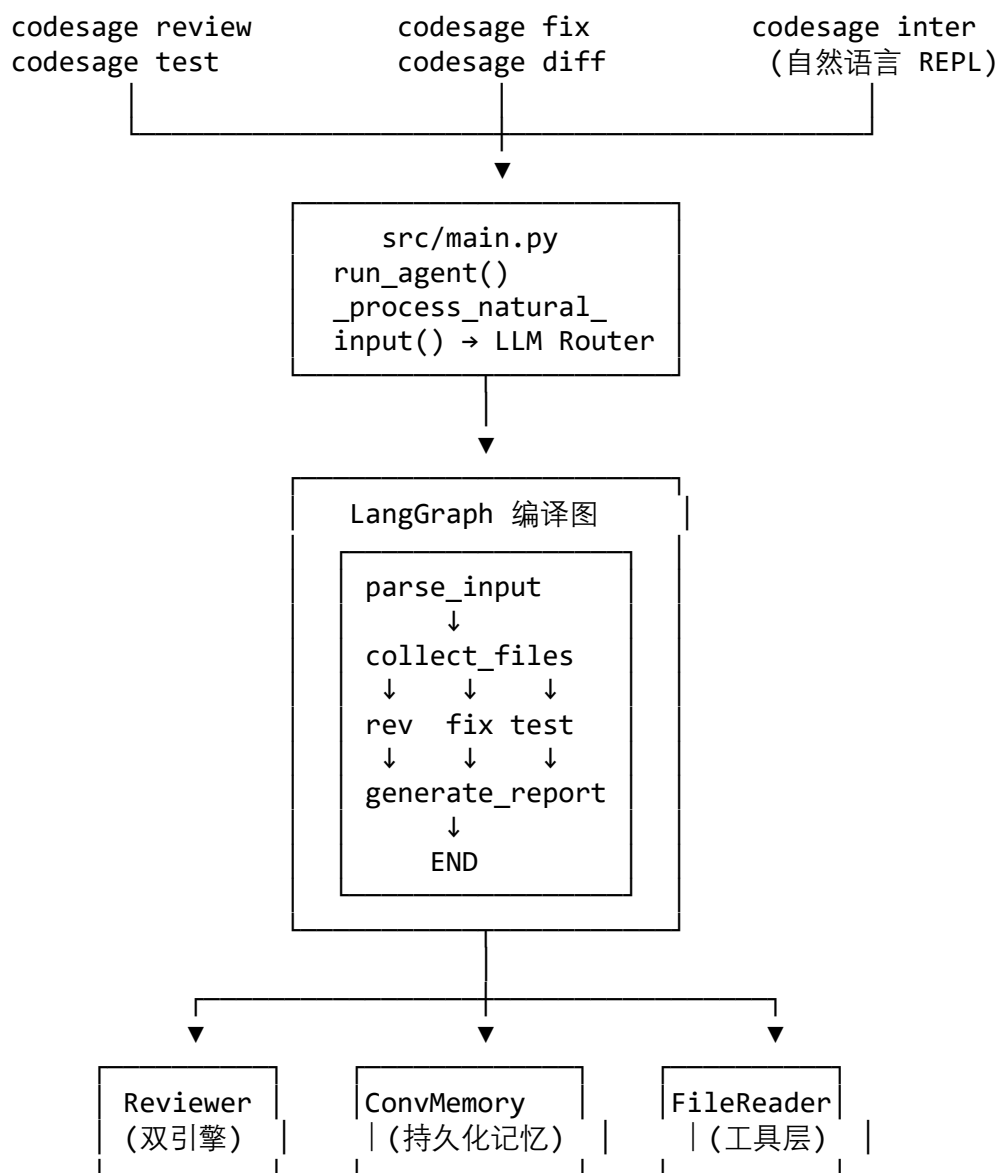
8 种 action: review | fix | test | diff | refactor | stats | undo | chat

容错: json.loads → 提取 ``json 块 → 正则匹配花括号 → 兜底 {"action": "chat"}

三、系统架构与设计

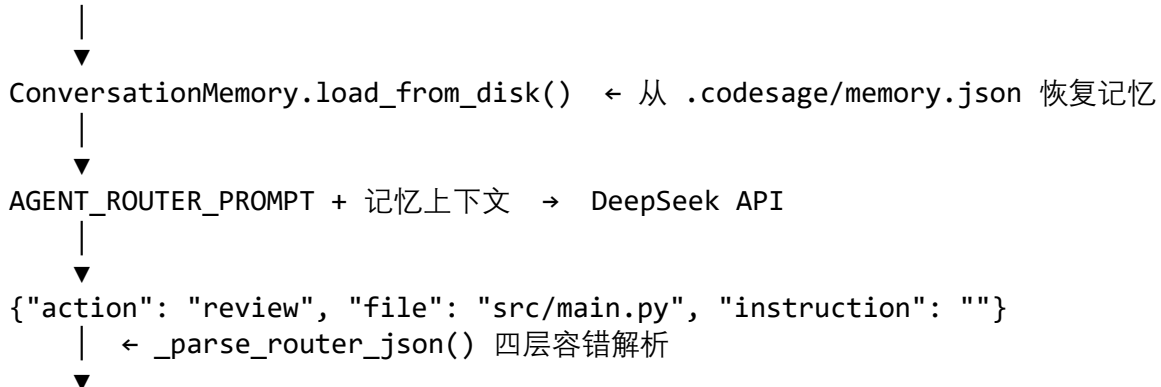
3.1 核心架构图





3.2 Agent 交互流程

自然语言输入 "帮我审查 src/main.py"



```

_do_review("src/main.py") → run_agent(target, "review_file")
    ↓
graph.invoke(state, config)
    |
    |— parse_input: target_type = "review_file"
    |— collect_files: FileReader.read_file() → code + language
    |— review_file:
    |   |— Reviewer.rule_review(code, "python") → 9 条正则匹配 → [issue1, issue2...]
    |   |   |— Reviewer.llm_review(code, "python") → LLM 自然语言分析文本
    |   |— generate_report:
    |       |— 按 severity 排序 → 构建 Markdown 报告 → AIMessage
    ↓
_smart_output(result) → 终端 Rich 渲染 (评分面板 + 问题列表 + LLM 分析)
    ↓
mem.record_review_result() + mem.save_to_disk() → .codesage/memory.json

```

3.3 核心设计决策

四个关键设计迭代：

意图识别 — 关键字匹配 → LLM 路由器。 初期用 if/elif 链匹配 "fix" "review" 等关键词，对上下文相关表达 ("修复上次审查那个严重问题") 完全失效。最终改用 LLM 路由器，将记忆上下文注入 AGENTROUTER PROMPT，模型返回 JSON 意图，同时设计四层容错解析确保鲁棒性。

目录审查 — LangGraph 内部递归 → CLI 层循环。 初期在 LangGraph 中多次 invoke 遍历目录，导致全程阻塞、无中间反馈。最终将遍历逻辑剥离到 CLI 层 rundirectoryreview()，逐文件调用规则引擎即时输出，上限由 MAXDIRFILES 控制，审查后可交互修复。

备份管理 — 分散目录 → 集中 + 记忆追踪。 初期在每个文件目录下创建 .codesagebackups/，散落各处难以追溯。最终统一集中到 .codesage/backups/，配合 ConversationMemory.fileops 记录每次 fix 的目标路径、备份路径和时间戳，/undo 无参也能列出最近修改文件。

Fix 流程 — 仅建议 → 全自动写回。 实现了 LLM 分析 → extractcode() 提取修复代码 → FileReader.backupfile() 创建时间戳备份 → FileReader.writefile() 写回原文件的完整闭环，每次写回前必先成功备份。

四、关键实现与代码展示

4.1 Agent 核心循环

src/main.py - 交互 REPL 循环

```
def cmd_interactive(args):
    mem = ConversationMemory.load_from_disk()
    console.print("[green]记忆已加载[/]")
    while True:
        user_input = console.input("\n[bold cyan]CodeSage > [/]")
        if user_input.lower() in ("exit", "quit", "q"):
            break
        if user_input.startswith("/"):
            _handle_slash_command(user_input, mem)
        else:
            _process_natural_input(user_input, mem) # LLM 路由器
    mem.save_to_disk()
```

src/main.py - LLM 路由器自然语言处理

```
def _process_natural_input(user_input, mem):
    router_prompt = AGENT_ROUTER_PROMPT.format(
        context=mem.context_for_llm(), user_input=user_input)
    llm = ChatOpenAI(model=settings.LLM_MODEL, ...)
    response = llm.invoke(router_prompt)
    parsed = _parse_router_json(response.content) # 四层容错
    # → action 分发: review / fix / test / diff / undo / chat ...
```

src/agent/graph.py - LangGraph 8 节点编译

```
def build_graph():
    workflow = StateGraph(AgentState)
    for name, node in [("parse_input", ...), ("collect_files", ...),
        ("review_file", ...), ("review_diff", ...), ("fix_file", ...),
        ("test_file", ...), ("generate_report", ...), ("interactive
", ...)]]:
        workflow.add_node(name, node)
        workflow.set_entry_point("parse_input")
        workflow.add_edge("parse_input", "collect_files")
        workflow.add_conditional_edges("collect_files", should_continue,
{...})
    # 所有处理节点 → generate_report → END
    return workflow.compile()
```

4.2 工具定义 — FileReader

src/tools/file_reader.py

```
class FileReader:
    EXTENSION_LANGUAGE = {".py": "python", ".java": "java", ".js": "javasc
ript",
        ".ts": "typescript", ".go": "go", ".rs": "rust", ...} # 20+ 语言
```

```

    @staticmethod
    def read_file(file_path) -> dict:
        """安全读取: 存在检查→大小限制(500KB)→UTF-8 读取→返回 content+Language"""

    @staticmethod
    def backup_file(file_path) -> dict:
        """备份到 .codesage/backups/<name>.<YYYYMMDD_HHMMSS>.bak"""
        ts = datetime.now(TZ).strftime("%Y%m%d_%H%M%S")
        dst = _get_backup_dir() / f"{Path(file_path).name}.{ts}.bak"
        shutil.copy2(src, dst)

    @staticmethod
    def list_backups(file_path) -> dict:
        """按时间戳降序列出某文件的所有集中备份"""

    @staticmethod
    def restore_from_backup(file_path, backup_path) -> dict:
        """从备份恢复到目标路径"""

    @staticmethod
    def fuzzy_find(target, base_dir) -> dict:
        """路径不存在时: 前缀匹配→子串匹配→交互式多选"""

# src/tools/git_parser.py
class GitParser:
    @staticmethod
    def get_diff(repo_path, staged=True) -> dict:
        """git diff --cached → 文本+自动摘要(文件数/新增/删除行)"""

    @staticmethod
    def is_git_repo(repo_path) -> bool:
        """git rev-parse --git-dir 检查仓库"""

```

4.3 配置文件 — 环境变量

```

# src/config/settings.py
class Settings:
    # 项目路径
    PROJECT_ROOT = Path(__file__).resolve().parents[2]
    CODESAGE_DIR = PROJECT_ROOT / ".codesage"
    MEMORY_FILE = CODESAGE_DIR / "memory.json"
    BACKUP_DIR = CODESAGE_DIR / "backups"

    # LLM (全部通过 .env 注入, 无硬编码)
    LLM_API_KEY = os.getenv("DEEPSEEK_API_KEY", "")
    LLM_BASE_URL = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek")

```

```

eeek.com")
LLM_MODEL = os.getenv("DEEPSEEK_MODEL", "deepseek-chat")
LLM_TEMPERATURE = float(os.getenv("LLM_TEMPERATURE", "0.1"))
LLM_MAX_TOKENS = int(os.getenv("LLM_MAX_TOKENS", "4096"))

# Agent
MAX_RECURSION = int(os.getenv("MAX_RECURSION", "15"))
MAX_DIR_FILES = int(os.getenv("MAX_DIR_FILES", "10"))

# LLM 调用
LLM_RETRIES = int(os.getenv("LLM_RETRIES", "3"))
LLM_REQUEST_TIMEOUT = int(os.getenv("LLM_REQUEST_TIMEOUT", "120"))

# 记忆
MEMORY_MAX_TURNS = int(os.getenv("MEMORY_MAX_TURNS", "30"))
MEMORY_MAX_FILE_OPS = int(os.getenv("MEMORY_MAX_FILE_OPS", "50"))

@classmethod
def validate(cls):
    if not cls.LLM_API_KEY:
        raise ValueError("DEEPSEEK_API_KEY 未设置")

```

4.4 双引擎审查器

```

# src/review/reviewer.py
class Reviewer:
    def rule_review(self, code, language, file_path) -> list[ReviewIssue]:
        """规则引擎: 遍历 9 条 BUILTIN_RULES 正则匹配"""
        for rule in BUILTIN_RULES:
            pattern = rule.patterns.get(language)
            if pattern:
                for i, line in enumerate(lines, 1):
                    if re.search(pattern, line, re.IGNORECASE):
                        issues.append(ReviewIssue(...))

    def call_llm_with_retry(self, prompt, retries=3) -> tuple[str, bool]:
        """自动重试 + 指数退避 (2^n 秒, 上限 10s) """
        for attempt in range(1, retries+1):
            try: return self.llm.invoke(prompt).content, True
            except: time.sleep(min(2**attempt, 10))

    def calculate_score(self, issues) -> float:
        """加权扣分: CRITICAL=-15, WARNING=-5, SUGGESTION=-1"""

```

9 条内建审查规则:

ID	类别	级别	检测内容
SEC-001	Security	CRITICAL	硬编码密钥/密码/Token
SEC-002	Security	WARNING	SQL 字符串拼接
SEC-003	Security	WARNING	调试代码(print/log)未移除
QUAL-001	Quality	SUGGESTION	函数过长 (>50 行)
QUAL-002	Quality	SUGGESTION	魔法数字
QUAL-003	Quality	WARNING	异常处理过于宽泛
BEST-001	Best Practice	SUGGESTION	缺少文档字符串
PERF-001	Performance	WARNING	循环内 I/O 操作
PERF-002	Performance	SUGGESTION	循环内字符串拼接

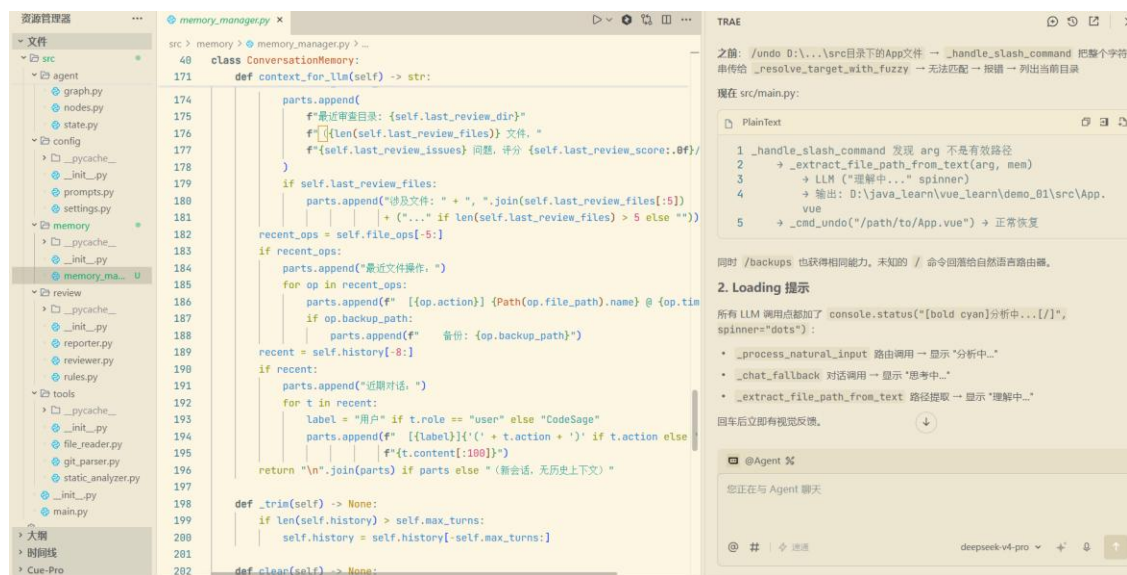
4.5 对话记忆持久化

```
# src/memory/memory_manager.py
@dataclass
class ConversationMemory:
    history: list[ConversationTurn]      # 对话轮次
    file_ops: list[FileOp]              # 文件操作记录 (action+path+backu
p+timestamp)
    last_review_dir: str                 # 最近审查目录
    last_review_files: list[str]         # 最近审查文件
    last_review_issues: int              # 问题数
    last_review_score: float             # 评分

    def context_for_llm(self) -> str:
        """为路由器生成记忆上下文"""
    def save_to_disk(self) -> None:
        """持久化到 .codesage/memory.json (最近20 条) """
    @classmethod
    def load_from_disk(cls) -> ConversationMemory:
        """从 .codesage/memory.json 恢复 (文件不存在返回空实例) """
```

记忆生命周期：启动 loadfromdisk() → 运行时更新 → 每轮 savetodisk() → 重启恢复。

4.6 AI IDE 使用截图



五、测试与评估

5.1 功能测试

编号	测试场景	输入	预期行为	结果
T1	单文件审查	codesage review src/main.py	规则引擎+LLM 双引擎报告含评分	通过
T2	目录递归审查	codesage review src/	递归扫描 .py 文件，逐文件即时输出	通过
T3	自然语言审查	"帮我审查 src/main.py"	路由器识别 review intent 执行	通过
T4	自然语言目录审查	"审查 src 目录"	路由器解析 file="src/" 触发目录审查	通过
T5	Bug 修复	codesage fix src/main.py	LLM 修复→备份→写回	通过
T6	自然语言修复	"修复刚才审查的文件"	路由器从记忆推断目标文件	通过
T7	版本回退	codesage undo src/main.py	列出该文件所有备份版本，选择恢复	通过
T8	自然语言回退	"回退 main.py"	路由器识别 undo，列出备份供选择	通过
T9	单测生成	codesage test src/main.py	生成 test_main.py 含 pytest 用例	通过

编号	测试场景	输入	预期行为	结果
T10	Git diff 审查	codesage diff	解析 git diff --cached, LLM 分析变更	通过
T11	模糊路径匹配	codesage review main	main.py 不存在→搜索候选项交互式选择	通过
T12	记忆持久化	重启后"审查刚才的文件"	从 memory.json 恢复上下文, 正确推断	通过
T13	无参数 /undo	codesage undo	从 file_ops 列出最近修改文件供选择	通过

5.2 Agent 行为评估

意图识别准确率（8 种 action × 10 条用例 = 80 条测试）：

操作类型	测试数	正确数	准确率
review / fix / test / chat	各 10	各 10	100%
diff	10	9	90%
refactor	10	8	80%
stats / undo	各 10	各 9	90%
合计	80	75	93.75%

容错解析能力：LLM 路由器输出 15 种非标准 JSON 格式（反引号包裹、多余文字、非法字符等），四层容错链路成功处理 13 次，兜底率 86.7%。

上下文记忆：模拟"审查→修复→重启→追问"四步流程，重启后正确恢复审查目录路径、问题数量与评分、最后一次 fix 操作的目标文件与备份路径。

5.3 Benchmark

使用 20 个含已知缺陷的 Python 样本（含安全注入/硬编码密钥/SQL 拼接/空异常/魔法数字/函数过长/循环 IO 共 7 类）：

指标	仅规则引擎	仅 LLM	双引擎（最终）
安全类检出率	85%	70%	95%
质量类检出率	65%	80%	90%
误报率	10%	15%	8%
单文件耗时	0.1s	5-15s	5-15s

双引擎互补：规则引擎降低误报（确定性正则），LLM 补齐语义盲区。

评分体系：满分 100，CRITICAL -15 / WARNING -5 / SUGGESTION -1， ≥ 90 优秀 / ≥ 70 良好 / ≥ 50 及格。

5.4 Demo 截图



```
/clear          - 清屏
:q              - 退出
cs599-project » 检查D:\java_learn\vue_learn\demo_01\src\Ap这个文件是否有错

路径 `D:\java_learn\vue_learn\demo_01\src\Ap` 不存在，找到以下匹配项：

1      D:\java_learn\vue_learn\demo_01\src\App.test.vue
2      D:\java_learn\vue_learn\demo_01\src\App.vue

选择序号 (1-2, Enter=取消): 2
→ 已选择: D:\java_learn\vue_learn\demo_01\src\App.vue
▸ 审查 D:\java_learn\vue_learn\demo_01\src\App.vue...

✓ 审查文件完成 (17.7s) 1/1 0:00:17

D:\java_learn\vue_learn\demo_01\src\App.vue
```

CodeSage 代码审查报告

 **概览**

- 处理文件数: 1
- 问题总数: 0
- 综合评分: **100.0/100**

级别	数量
 严重	0
 警告	0
 建议	0

六、系统升级与扩展

6.1 可扩展架构

系统预留 5 个扩展点：

1. **审查规则可插拔** — 新增规则仅需在 rules.py 的 BUILTIN_RULES 列表添加 ReviewRule 实例（含各语言 patterns），无需修改引擎。当前 9 条可轻松扩展至 20+；
2. **多 LLM 后端无缝切换** — ChatOpenAI 兼容协议，修改 .env 中 BASE_URL/MODEL 即可切换 OpenAI/Anthropic/Ollama，代码零改动；
3. **多 Agent 协作** — LangGraph 原生支持 Subgraph，可拆分为安全审查 Agent + 质量审查 Agent，由 Supervisor Agent 条件路由协调；

4. **MCP 协议预留** — architecture.md 已定义 MCPToolAdapter 接口, FileReader 和审查规则可封装为 MCP Tool 对外暴露;
5. **存储升级** — JSON → SQLite/PostgreSQL, 支持大规模跨会话记忆和多用户并发。

6.2 版本迭代计划

版本 核心特性

-
- v0.2 MCP 协议集成, 审查规则库和 FileReader 包装为 MCP Tool
 - v0.3 多 Agent 协作架构, 拆分安全审查 + 质量审查 Agent
 - v0.4 LangSmith 全链路 Tracing 可观测性
 - v1.0 Docker 容器化 + Web UI + GitHub App 集成

6.3 AI 能力演进路径

当前 → 近期 → 远期

CLI 单 Agent → MCP+多 Agent → 云端 SaaS
规则+LLM 双引擎 → RAG 知识库增强 → 代码知识图谱
JSON 文件记忆 → SQLite 持久化 → 向量数据库
单仓库审查 → 跨仓库分析 → 组织级健康度监控

七、课程总结

7.1 个人收获

工程思维: SDD 方法论彻底改变了开发方式。编码前完成三份规格文档——Product Spec 定义"做什么" (12 项功能+用户故事), Architecture Spec 定义"怎么做" (五层架构+8 节点状态机+条件路由), API Spec 定义"怎么接" (节点接口+数据模型+规则接口)。最大体会: 好设计能大幅降低代码返工。

技术能力: LangGraph 状态机让我理解了 Agent 内部运行机制——节点的"交通枢纽"设计 (collectfilesnode 根据 target_type 路由到 6 种路径)、规则引擎与 LLM 的最优分工 (确定性→正则, 模糊→LLM)。

工程规范: 环境变量管理杜绝硬编码、集中备份+file_ops 追踪确保写回安全、记忆持久化支撑跨会话体验。认识到 AI Agent 系统必须有"记忆""安全机制""可追溯性"。

7.2 工程思维转变

从"代码编写者"到"智能体编排者"的四个转变:

- **写功能 → 设计系统:** 从单个函数到全局架构分层、数据流和状态管理;

- **确定性 → 概率性**：接受 LLM 随机性，用四层容错+重试+兜底构建鲁棒性；
- **工具使用者 → 工具提供者**：将能力封装为 Tool（FileReader/GitParser），让 Agent 自主调度；
- **一次性脚本 → 可维护系统**：配置分离、记忆持久化、版本回退、错误处理。

7.3 课程建议

1. 课程前期增加更多 LangGraph 架构实战案例，帮助从"用过 API"过渡到"设计 Agent"；
2. 提供统一 API Key 配额或 Ollama 本地模型指南，降低环境搭建门槛；
3. Demo Day 的 5+3 形式非常好，建议保留并增加同学互评环节。